Project Interim Report

On

Automated Teller Machine (ATM) Simulation (Using Core Java & OOP Concepts)

RU-100-01-00012

Object Oriented Programming with Java

SESSION: 2025-26

Submitted By
Mayank Bhashkar
RU-25-11735

Bachelor of Technology in Computer Science & Engineering with AI and ML in association with GOOGLE

Under the Guidance of
MR. ASHISH SHIVHARE

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

RUNGTA INTERNATIONAL SKILLS UNIVERSITY

BHILAI, CG

(April 2026)

Abstract

The Automated Teller Machine (ATM) Simulation is a console-based Java application developed to replicate the fundamental operations of a real-world ATM system. In today's fast-paced banking environment, automated systems are essential to reduce human dependency, minimise errors, and deliver quick, secure services to customers. This project addresses that need by providing a simulated environment where a user can authenticate using a secure static PIN and carry out basic banking operations such as balance enquiry and cash withdrawal.

The system is built entirely using Core Java and demonstrates the practical application of Object-Oriented Programming (OOP) principles, including encapsulation, class design, object instantiation, static variables, and structured exception handling. The ATM class manages authentication and user interaction through a menu-driven interface, while the Account class is responsible for maintaining the account balance and processing transactions. Together, these classes create a clean, modular, and extensible simulation that is ideal for academic study. This project proves that even a fundamental system like an ATM can be effectively modelled using OOP concepts within a simple Java program, without any external libraries or databases.

Introduction

An Automated Teller Machine (ATM) is an electronic banking outlet that allows customers to complete basic transactions without the need for a human teller or bank representative. The concept was introduced to improve efficiency in banking services. This project, titled "Design and Implementation of an ATM Simulation using Object-Oriented Programming," brings the core logic of an ATM to life using Java, one of the most widely used object-oriented programming languages.

The project uses fundamental OOP concepts to design a simulation where a user interacts with a simple menu interface to check their account balance or withdraw cash after successful PIN verification. The use of encapsulation ensures that sensitive data such as the PIN and balance are hidden from direct access, and are only manipulated through well-defined methods. This reflects real-world security practices used in banking software.

Key Features

· PIN-based authentication with a limited number of attempts

· Real-time balance enquiry

· Cash withdrawal with balance validation

· Menu-driven user interface for easy interaction

· Error handling using Java exception handling mechanisms

Objectives

· Manage account information securely using encapsulation

· Perform financial transactions such as withdrawal and balance checking

· Reduce manual banking work through automation

· Apply and demonstrate core OOP concepts in a real-world scenario

· Develop a clean, modular Java program suitable for academic submission

Scope

The scope of this project is focused on simulating the basic operations of an Automated Teller Machine in a console-based Java environment. The current version of the system supports a single user account with a pre-set balance and a static PIN for authentication. The following are the key areas covered by this project:

· Suitable for small-scale banking simulation in an academic or educational context

· Demonstrates the use of OOP principles in Java effectively

· Can be extended in the future to include database connectivity for storing multiple user accounts

· Can be upgraded to include a Graphical User Interface (GUI) using Java Swing or JavaFX

· A deposit feature and transaction history module can be added as an extension

· Multi-user support with unique PINs and account numbers can be introduced with a database backend

System Requirements

Hardware Requirements

· Processor: Intel Core i3 or equivalent (minimum)

· RAM: 4 GB (minimum)
· Storage: 500 MB of free disk space
· Input Devices: Keyboard and Mouse

Software Requirements

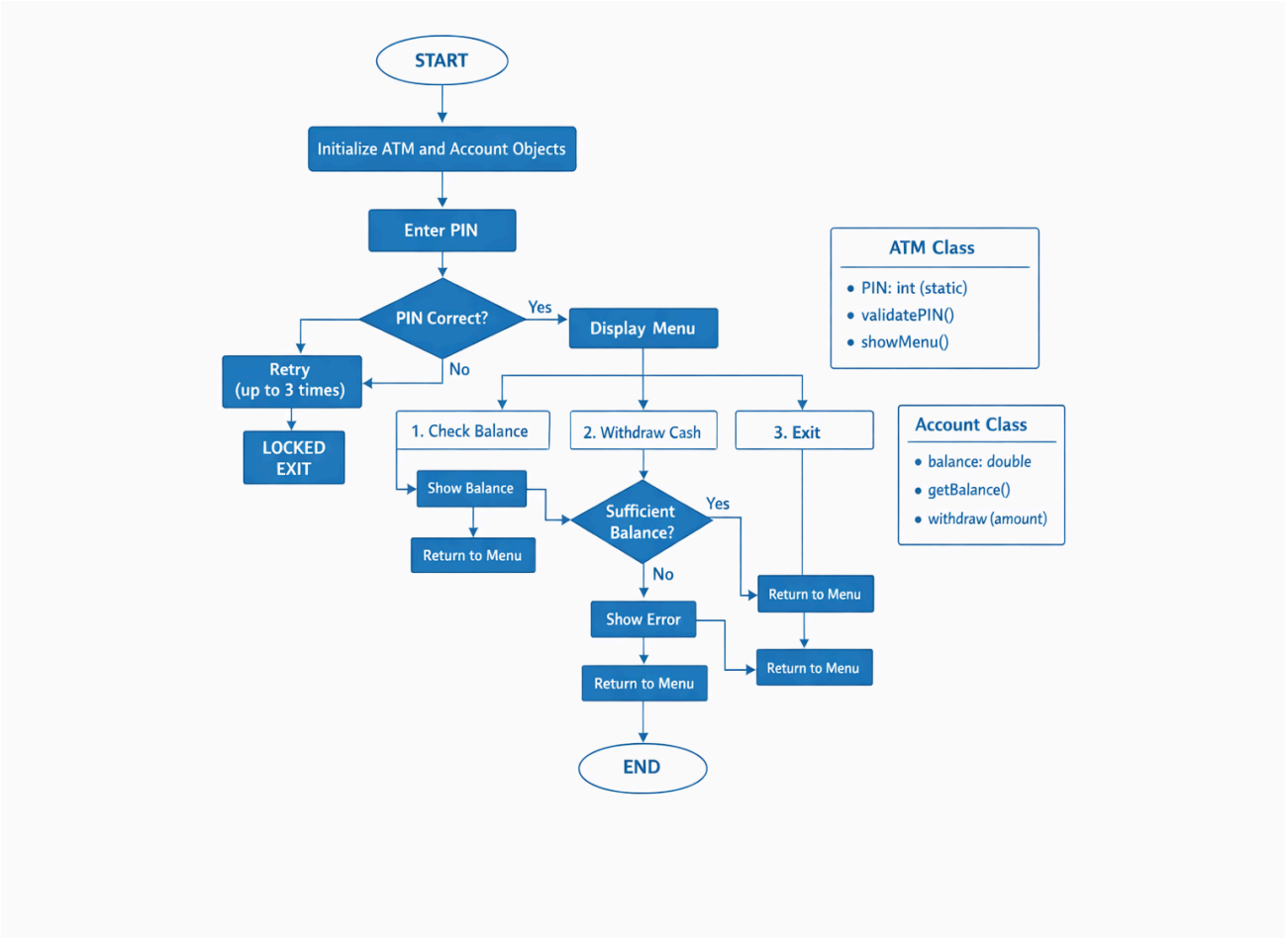
· Operating System: Windows 10 / 11, Linux, or macOS
· Programming Language: Java (JDK 11 or above)
· IDE: VS Code, Eclipse, IntelliJ IDEA, or EditPlus
· Java Runtime Environment (JRE) installed
· Command Prompt or Terminal for program execution

Methodology

The ATM Simulation follows a step-by-step sequential process to carry out transactions. The methodology is structured as follows:

· Step 1 – System Start: The program begins execution and initialises the ATM object with a pre-defined PIN and an Account object containing the starting balance.
· Step 2 – PIN Entry: The user is prompted to enter their 4-digit PIN. The system checks the entered PIN against the stored PIN. If it matches, access is granted. If incorrect, the system allows up to three attempts before locking the session.
· Step 3 – Main Menu Display: Upon successful authentication, the user is presented with a menu containing three options: (1) Check Balance, (2) Withdraw Cash, (3) Exit.
· Step 4 – Balance Enquiry: If the user selects option 1, the current account balance is fetched from the Account object using the getBalance() method and displayed on the console.
· Step 5 – Cash Withdrawal: If the user selects option 2, they are prompted to enter the amount to withdraw. The system validates whether sufficient balance exists. If yes, the amount is deducted using the withdraw() method. If no, an error message is shown.
· Step 6 – Exit: When the user selects option 3, the session ends gracefully with a thank-you message. The loop exits and the program terminates.

Flowchart



Implementation

The implementation is divided into two Java classes: ATM and Account. Both classes are placed within the same package and work together to simulate the ATM's core functionality.

Account Class Logic

The Account class is designed with the principle of encapsulation. The balance field is declared as private so it cannot be accessed directly from outside the class. The getBalance() method provides read-only access to the balance, and the withdraw() method handles all withdrawal logic. Inside withdraw(), the system checks if the requested amount is greater than zero and does not exceed the current balance. If both conditions are met, the amount is subtracted and the method returns true to indicate success. Otherwise, it returns false.

ATM Class Logic

The ATM class serves as the main controller. A static integer variable PIN is set to the correct PIN value at the class level. The main() method creates an Account object with an initial balance. A loop then repeatedly prompts the user for their PIN, allowing up to three attempts. Once authenticated, the showMenu() method presents the transaction menu using a while loop that continues until the user chooses to exit. A Scanner object handles all user input. Exception handling ensures that non-integer inputs do not crash the program.

Menu-Driven Program

The menu is implemented using a switch-case statement inside a while loop. The user can repeatedly perform operations until they opt to exit. Each operation calls the appropriate method of the Account class and displays the result on the console.

Sample Input and Output

The following is a sample interaction with the ATM Simulation:

--- ATM Machine ---

Enter your PIN: 1234

Access Granted!

1. Check Balance

2. Withdraw Cash

3. Exit

Enter your choice: 1

Your current balance is: Rs. 10000.0

Enter your choice: 2

Enter withdrawal amount: 3000

Withdrawal successful! Amount withdrawn: Rs. 3000.0

Remaining Balance: Rs. 7000.0

Enter your choice: 3

Thank you for using our ATM. Goodbye!

Future Enhancements

While the current version of the ATM Simulation fulfils its academic purpose, there are several improvements that can be made to enhance its functionality and bring it closer to a production-level application:

· Graphical User Interface (GUI): The console-based interface can be replaced with a graphical interface using Java Swing or JavaFX, making the system more user-friendly and visually appealing.

· Database Integration: The system can be connected to a relational database such as MySQL to store multiple user accounts, transaction history, and dynamic balance information persistently.

· Multiple User Support: The current system supports only a single user. A login system with unique account numbers and PINs for multiple users can be implemented using arrays, ArrayLists, or database tables.

· Deposit Feature: A cash deposit functionality can be added, allowing users to deposit money into their account from the ATM interface.

· Transaction History: Each withdrawal and deposit can be recorded and displayed to the user as a mini statement or transaction log.
· PIN Change Option: An option to change the PIN securely after verifying the current PIN can improve account security.
· Time-Out Feature: The system can automatically log out after a period of inactivity to prevent unauthorised access.

Gantt Chart

A Gantt Chart is a visual project management tool used to plan and track the timeline of tasks involved in a project. Below is a description of the phases and their estimated durations for the ATM Simulation project:

Phase 1: Planning and Requirement Analysis (Week 1)

During this phase, the project objectives, scope, and requirements were identified. The features to be included (PIN authentication, balance check, withdrawal) were defined. The basic design of the system architecture was also planned.

Phase 2: System Design (Week 2)

The class structure was finalised. The ATM class and Account class were designed with their attributes and methods. A flowchart was drawn to represent the logical flow of the program, and the class diagram was drafted.

Phase 3: Implementation and Coding (Weeks 3–4)

The Java code was written based on the design. The Account class was implemented first, followed by the ATM class. The menu-driven logic, PIN validation, and exception handling were coded and compiled.

Phase 4: Testing and Debugging (Week 5)

The program was tested for various scenarios, including correct PIN entry, incorrect PIN entry, valid withdrawal, insufficient balance, and invalid input. Bugs were identified and fixed during this phase.

Phase 5: Documentation and Report Writing (Week 6)

The project report was prepared, including all sections: Abstract, Introduction, Scope, Methodology, Flowchart, Implementation, Future Enhancements, Gantt Chart, Conclusion, and References.

Conclusion

The ATM Simulation project has been successfully designed and implemented using Core Java and Object-Oriented Programming principles. The project achieves all its defined objectives: it simulates the key operations of a real ATM, applies encapsulation to protect sensitive data, uses classes and objects effectively, and delivers a clean menu-driven user experience through the console.

The use of OOP concepts such as encapsulation, class design, static variables, and exception handling demonstrates a strong understanding of Java programming at the B.Tech level. The separation of responsibilities between the ATM class (authentication and menu control) and the Account class (balance management and transactions) reflects good software design practices.

Overall, this project is a valuable learning exercise that bridges the gap between theory and practical application of OOP in Java. It lays a solid foundation for future enhancements such as GUI development, database integration, and multi-user support, making it a scalable and meaningful academic project.

References

Books

[1] H. Schildt, Java: The Complete Reference, 11th ed. New York: McGraw-Hill Education, 2018.

[2] B. Eckel, Thinking in Java, 4th ed. Upper Saddle River, NJ: Prentice Hall, 2006.

Websites

[3] Oracle, “Java Documentation – The Java Tutorials,” Oracle. [Online]. Available: https://docs.oracle.com/javase/tutorial/

[4] GeeksforGeeks, “Object Oriented Programming in Java,” GeeksforGeeks. [Online]. Available: https://www.geeksforgeeks.org/object-oriented-programming-oops-concept-in-java/